

MUHAMMAD GHULAM JILLANI

Lead AI Engineer / Generative AI, Agentic Systems and Cloud Architecture

Agentic RAG · Multi-Agent Systems · Model Fine-Tuning · LLMOps · Production AI on AWS, Azure and GCP

m.g.jillani123@gmail.com | +1-501-420-2439 | LinkedIn | Upwork (Top Rated Plus) | GitHub | Kaggle | Medium | Fully remote · USA, UK, Germany, EU

I build production generative AI systems and stay with them after launch. 6 years and 22+ shipped systems across healthcare, legal, financial services, retail and SaaS: agentic RAG over private enterprise data, multi-agent workflows, open-weight models fine-tuned for domain accuracy, and the evaluation and observability layer that keeps quality and token cost under control at scale. I deploy on AWS, Azure or GCP depending on where the client already runs, and I work from the architecture decision rather than the demo, which includes saying when a smaller classical model beats a large language model on accuracy, latency and cost.

6+	22+	27+	\$870K+	47%	99.9%
YEARS EXPERIENCE	PRODUCTION SYSTEMS	ENTERPRISE CLIENTS	CLIENT SAVINGS	AVG EFFICIENCY GAIN	PRODUCTION UPTIME

CAPABILITY MAP

GENERATIVE AI AND AGENTS

Agentic RAG	Multi-Agent Systems	GraphRAG	Tool Calling
Structured Output	Guardrails	Evaluation Pipelines	MCP
Computer-Use Agents			

Model routing and fallback across Claude, GPT, Gemini and open-weight models, tool-calling agents with human approval gates, and guardrails for PII redaction, topic policy and jailbreak filtering.

RETRIEVAL AND DATA

Hybrid Vector + BM25	Re-Ranking	Knowledge Graphs	Chunking Strategy
Ingestion Pipelines	Multimodal		

Retrieval built around the actual data: precision measured against a labeled set, role-based filtering applied at query time, and re-embedding triggered on source change.

MODEL TRAINING AND SERVING

Fine-Tuning	QLoRA and LoRA	DPO Alignment	Distillation
Quantization	vLLM	GPU Capacity Planning	

Domain tuning of open-weight models when frontier APIs are too costly or too generic, with evaluation gates wired into promotion so no checkpoint reaches production untested.

CLOUD, MLOPS AND DELIVERY

AWS	Bedrock	SageMaker	Lambda	Step Functions	Azure OpenAI
Vertex AI	Kubernetes	CI/CD	Observability		

AWS is my primary platform and I hold the certification, with Azure and GCP used where the client already runs there. VPC-isolated deployments for regulated data, least-privilege access, and telemetry on latency, drift and per-feature token spend.

CERTIFICATIONS

Claude Certified Architect, Foundations | Anthropic

Verify: credly.com/badges/d7d06be6-68a7-4790-bd3d-d2cfd319ffa0

- AWS Certified Solutions Architect
 - AWS Cloud Solutions Architect Professional Certificate, Amazon Web Services
 - Practical Data Science on the AWS Cloud Specialization, DeepLearning.AI
 - Microsoft Azure AI Fundamentals (AI-900), Microsoft
 - Preparing for Google Cloud Certification: Machine Learning Engineer, Google
 - MLOps, Machine Learning Operations Specialization, Duke University
- Machine Learning Engineering for Production (MLOps), DeepLearning.AI
 - Large Language Model Operations (LLMOps), Duke University
 - Deep Learning Specialization, DeepLearning.AI
 - IBM Machine Learning Specialization, IBM
 - Advanced Machine Learning on Google Cloud Specialization, Google Cloud
 - Google Advanced Data Analytics Professional Certificate, Google

HOW I WORK

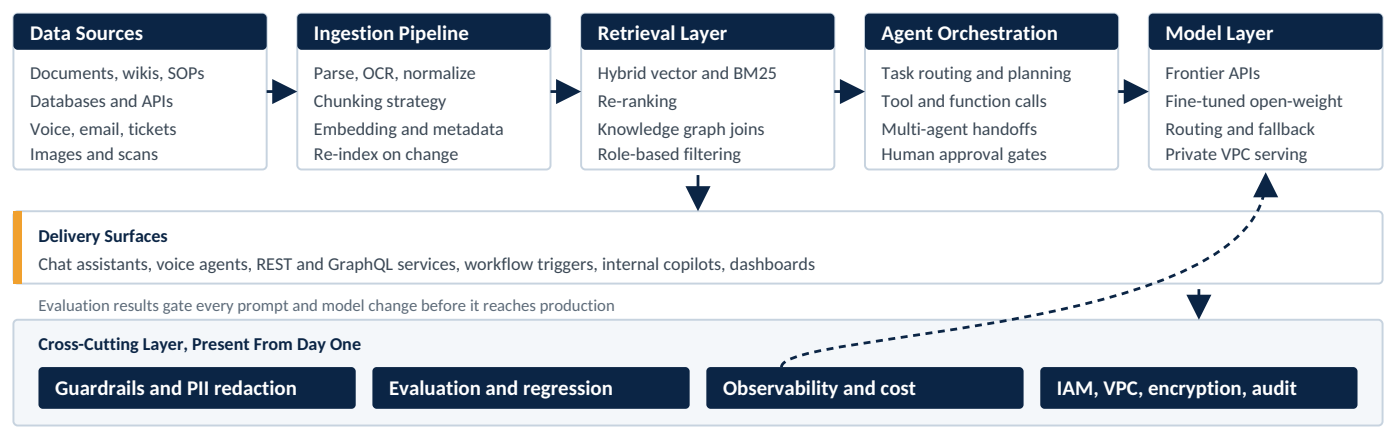
- **Architecture before code.** Model choice, retrieval design, latency budget and cost per request get decided up front, not discovered in production.
 - **Evaluation gates.** Every system ships with a measured baseline and a regression suite, so a prompt or model change cannot quietly degrade quality.
- **Right-sized solutions.** Generative AI where it wins, classical machine learning where it is cheaper, faster and more accurate. I will tell you which one you need.
 - **Ownership after launch.** Monitoring, drift detection, cost governance and iteration once the system is live, not a handover and a disappearance.

INDUSTRIES

Healthcare and Clinical	Legal	Financial Services	Retail and E-commerce	SaaS and Product	Compliance and RegTech	Manufacturing and Operations
Customer Support and Contact Centers						

REFERENCE ARCHITECTURE: HOW I BUILD A PRODUCTION AI SYSTEM

The pattern below is what I deploy and adapt per client. It is cloud-portable: Bedrock, SageMaker and OpenSearch on AWS, Azure OpenAI and AI Search on Azure, or Vertex AI on GCP.



PROJECTS

Voice AI Agent Platform for Inbound Customer Operations VOICE AI

Problem: A high-volume support operation was losing tier-1 calls to hold queues, and every routine password reset or order status check consumed a human agent slot at full cost.

Solution: Real-time voice agent with streaming speech-to-text, Bedrock-hosted reasoning, barge-in handling and a strict tool layer against the client CRM and order systems. Latency budgeted end to end so reasoning, retrieval and synthesis fit under one second, with confidence-scored escalation that hands ambiguous calls to a human with the transcript attached.

Result: 80%+ of tier-1 calls handled autonomously, sub-1s response latency, 45% reduction in average handle time.

Stack: Bedrock, Transcribe, Polly, Lambda, API Gateway WebSockets, ECS Fargate, DynamoDB, CloudWatch, Python, FastAPI.

Clinical Decision Support System, US Hospital Network HIPAA

Problem: Clinicians were reconstructing patient context by hand from fragmented histories, lab reports and unstructured notes, and every minute spent doing that came out of patient intake.

Solution: HIPAA-compliant agentic RAG pipeline over patient records and current medical literature, with ICD-10 mapping, citation-bound answers and a refusal path when evidence is thin. PHI never leaves the client VPC: KMS encryption, private endpoints and audit logging on every retrieval and generation call.

Result: 91% factual accuracy on the clinical evaluation set, 27% less patient intake time, 48% less manual documentation.

Stack: Bedrock (Claude), OpenSearch Serverless, S3, KMS, PrivateLink, LangGraph, FastAPI, clinician-labeled gold evaluation set.

Domain-Tuned Legal LLM, 400K Document Fine-Tune FINE-TUNING

Problem: A legal team could not use general-purpose models because they invented citations, and frontier API pricing made high-volume clause review economically unworkable.

Solution: Fine-tuned Gemma 3 27B on a 400K document legal corpus with QLoRA, then applied DPO alignment targeting citation fidelity and abstention behavior. Data pipeline, training runs, evaluation gates and serving all built on SageMaker.

Result: 91% citation accuracy with zero hallucinated citations after alignment, 60% lower inference cost than the frontier API baseline.

Stack: SageMaker training and endpoints, S3, Gemma 3 27B, QLoRA, DPO, vLLM, MLflow, Python.

RegTech Compliance Engine for Multi-Jurisdiction ESG Reporting AGENTIC RAG

Problem: Compliance analysts were reading corporate disclosures line by line against shifting regulatory texts in three languages, and each reporting cycle burned weeks of specialist time.

Solution: Retrieval and classification engine that maps disclosure content against regulatory requirements across 8 jurisdictions, flags gaps with the source clause attached, and produces a reviewable audit trail behind every verdict.

Result: 84% gap classification accuracy, 52% less manual ESG auditing per cycle, 2.3x faster regulatory reporting across EN, DE and FR.

Stack: Bedrock, OpenSearch, Textract, S3, Step Functions, LangChain, FastAPI, Streamlit.

Enterprise Knowledge Search over Fragmented Documentation **GRAPHRAG**

Problem: Policies, SOPs, HR guidance and engineering manuals lived in four systems with no shared vocabulary, so employees raised tickets instead of finding answers.

Solution: Hybrid retrieval combining vector search, BM25 and a knowledge graph layer for entity and relationship queries that pure embedding search fails on, with metadata tagging, role-based filtering applied at retrieval time and a re-embedding pipeline triggered by document updates.

Result: 54% less internal search time, 61% better retrieval precision, 37% fewer knowledge-gap tickets, 8,000+ queries daily.

Stack: OpenSearch Serverless hybrid search, Bedrock, S3, Lambda, Neo4j, LangGraph, role-based access control.

Autonomous Revenue Operations Platform **MULTI-AGENT**

Problem: A sales organization was manually researching, scoring and sequencing outreach, which capped how many qualified conversations the team could create per week.

Solution: Multi-agent system that enriches inbound and outbound leads, scores fit against ICP criteria, drafts context-specific outreach and fires event-driven follow-ups, with human approval gates on anything customer-facing and durable workflows so a failed step retries rather than dropping a lead.

Result: \$340K pipeline in the first 6 months, 1.8x more qualified leads, 48% less manual sales operations work, 900+ automated triggers daily.

Stack: Bedrock, Step Functions, Lambda, EventBridge, Aurora PostgreSQL with pgvector, n8n, CRM integrations.

LLMOps Governance Control Plane **LLMOPS**

Problem: Five generative AI systems had reached production independently, each with its own prompts, model versions and logging, and nobody could answer which model produced a given output last quarter.

Solution: Consolidated all five under one control plane with prompt and model versioning, automated drift detection on output quality, per-feature token spend tracking, and lineage capture built for SOC 2 evidence requests, with regression suites gating every prompt or model change.

Result: 5 production models under unified governance, automated drift detection, SOC 2 audit-ready lineage on every inference call.

Stack: Bedrock, CloudWatch, S3, MLflow, LangSmith, GitHub Actions, Docker, Python.

Private LLM Deployment Inside Client Infrastructure **DATA RESIDENCY**

Problem: A client with strict data residency obligations could not send a single record to a third-party API, which ruled out every hosted assistant on the market.

Solution: Open-weight models deployed on GPU infrastructure inside the client boundary with quantized serving for throughput, an OpenAI-compatible gateway so internal teams could integrate without rewrites, and hardware sized against measured load instead of vendor guidance.

Result: 100% data residency maintained inside the client VPC, production-grade latency, no external model calls.

Stack: EC2 GPU instances, VPC isolation, vLLM, quantized open-weight models, LiteLLM gateway, Docker, Prometheus and Grafana.

Retail Personalization and Demand Forecasting Engine **ML + GENAI**

Problem: Recommendations were static and inventory planning ran on last season's assumptions, so the client was overstocked and out of the items customers wanted at the same time.

Solution: Gradient-boosted forecasting layer combined with real-time behavioral signals and a generative layer for merchandising copy. The forecasting core stays classical on purpose: more accurate, cheaper and more explainable than a language model for this problem.

Result: 14% higher e-commerce conversion, 31% better inventory accuracy, 62% better demand forecasting, sub-100ms serving.

Stack: SageMaker, Bedrock, Aurora PostgreSQL with pgvector, Airflow, Scikit-learn, FastAPI.

Autonomous Multi-Channel Support Agent **SUPPORT AI**

Problem: Support volume across seven channels landed in one shared human queue, so response time depended on which channel a customer happened to use.

Solution: Unified agent with per-channel context handling, retrieval over the product knowledge base, action tools for order and account operations, and escalation thresholds calibrated against real transcripts.

Result: 61% of queries resolved without human escalation, 44% faster average response, 23% better engagement, across 7 channels.

Stack: Bedrock, Lambda, DynamoDB, OpenSearch, API Gateway, LangGraph, FastAPI.

AI-Assisted DevOps and Incident Intelligence **AIOPS**

Problem: Engineers were reading raw logs across services during incidents, and the same failure signatures were being rediagnosed from scratch every time.

Solution: Log and trace correlation with anomaly detection, retrieval over past incident write-ups, and ranked root-cause hypotheses surfaced inside the on-call workflow, not in a separate dashboard nobody opens.

Result: 38% less debugging and incident time, 29% faster deployment cycles.

Stack: CloudWatch, OpenSearch, Bedrock, Lambda, GitHub Actions, Docker, Python.

TECHNICAL SKILLS

AI ENGINEERING AND LLM SYSTEMS

MACHINE LEARNING AND DATA SCIENCE

Models: Claude 4 family, GPT-5 family, Gemini, Llama, Gemma, DeepSeek, Qwen

Frameworks: LangChain, LangGraph, LlamaIndex, LangSmith, CrewAI, AutoGen, n8n

Architectures: Agentic RAG, GraphRAG, multi-agent systems, tool calling, prompt and context engineering, evaluation pipelines

Training: QLoRA and LoRA fine-tuning, DPO alignment, distillation, quantization, vLLM serving

Retrieval: Pinecone, Weaviate, Qdrant, ChromaDB, FAISS, pgvector, OpenSearch, Neo4j knowledge graphs

CLOUD, MLOPS AND LLMOPS

AWS (primary): Bedrock, SageMaker, Lambda, Step Functions, ECS Fargate, EC2 GPU, S3, OpenSearch Serverless, Aurora with pgvector, API Gateway, CloudWatch, IAM, KMS, VPC

Azure: Azure OpenAI, Azure ML, AKS, Blob Storage

GCP: Vertex AI, Cloud Run, BigQuery ML

Practices: MLflow, ZenML, model and prompt versioning, drift detection, cost governance, CI/CD with GitHub Actions

Methods: Forecasting, classification, ranking and recommendation, anomaly detection, NLP, computer vision, feature engineering

Frameworks: Scikit-learn, PyTorch, TensorFlow, Hugging Face, XGBoost

Data: Pandas, NumPy, PySpark, PostgreSQL, Snowflake, BigQuery, Kafka, Airflow

Reliability: Explainability with SHAP, drift monitoring, model evaluation and benchmarking

ENGINEERING AND DELIVERY

Backend: Python, FastAPI, Flask, REST and GraphQL, microservices, Streamlit

Infrastructure: Docker, Kubernetes, infrastructure as code, observability with Prometheus and Grafana

Security: VPC isolation, least-privilege access, encryption at rest and in transit, PII redaction, audit logging

Working with teams: Technical leadership, architecture reviews, stakeholder alignment, documentation and handover

DELIVERY PROCESS

1. **Discovery.** What the workflow costs today, where the failure modes are, and whether AI is the right instrument at all.

2. **Architecture.** Model choice, retrieval design, latency budget, cost per request and data boundaries agreed before code.

3. **Evaluation set first.** A labeled set and a measured baseline, so improvement is provable.

4. **Build in slices.** A thin working path to production early, then depth, so stakeholders see real output in weeks.

5. **Harden.** Guardrails, access control, audit logging, load testing and cost telemetry before launch, not after an incident.

6. **Own it.** Monitoring, drift detection, cost governance, iteration and documented handover to your team.

PROBLEMS I GET HIRED TO SOLVE

"**Our knowledge is unsearchable.**" Content spread across wikis, drives and ticket systems, with no shared vocabulary and no reliable way to find an answer.

"**Support volume is outgrowing the team.**" Repetitive tier-1 calls, chats and tickets consuming specialist capacity that should sit on hard cases.

"**Frontier API costs will not survive our volume.**" Per-token pricing that works in a pilot and breaks at scale, usually solvable with fine-tuning, routing and caching.

"**We cannot send this data anywhere.**" Regulated or contractual constraints that rule out hosted assistants and require deployment inside your own boundary.

"**We have demos, not production.**" Prototypes that impress in a meeting but lack evaluation, guardrails, access control and the operations to survive real users.

"**Nobody knows what our AI costs or when it degrades.**" Multiple systems live with no shared versioning, drift detection, spend visibility or lineage.

WORKING TOGETHER

WHAT THE FIRST CALL COVERS

The workflow you want to change, what it costs today in hours or headcount, your volumes and accuracy bar, where the data lives, and any residency or compliance constraints.

You get a straight read on feasibility on that call, including when the honest answer is that a simpler system, or no AI at all, is the better investment.

WHAT YOU GET BEFORE COMMITTING

A written architecture and scope: components, model and retrieval choices, data boundaries, delivery milestones and how success will be measured.

Sending a sample of the real documents or a description of current volumes lets me scope it properly.

RECOGNITION

Top Rated Plus on Upwork with a 100% Job Success Score and \$100K+ earned across enterprise AI engagements. LinkedIn Top Voice in AI and Cloud ML (24x). Top 100 Global Kaggle Master. NVIDIA Developer Program contributor, GitHub Developer Program member and KaggleX BIPOC mentor.

Available for generative AI architecture, build and delivery engagements

m.g.jillani123@gmail.com | +1-501-420-2439 | LinkedIn | Upwork | GitHub | Fully remote across USA, UK, Germany and EU time zones